

PRÁCTICA 1

Contenedores asociativos en C++

Factor de ponderación: 4

Semana del 14 al 18 de septiembre 2026

1. Objetivos

El objetivo de la práctica es trabajar con algunas estructuras de datos básicas en C++ y que serán de especial relevancia a la hora de desarrollar las prácticas de la asignatura. En particular, nos centraremos en el uso de los contenedores asociativos de la biblioteca estándar de C++ (*Standard Template Library, STL*) [1]. Además de introducir estas estructuras de datos, se propone que el alumnado utilice este ejercicio para poner en práctica aspectos generales relacionados con el desarrollo de programas en C++. Algunos de estos principios que enfatizaremos en las prácticas de la asignatura serán los siguientes:

- Programación orientada a objetos: es fundamental identificar y definir clases y objetos que permitan modelar adecuadamente el escenario de trabajo que se plantea.
- Diseño modular: el programa debiera escribirse de modo que las diferentes funcionalidades que se precisen sean encapsuladas en métodos concretos cuya extensión textual se mantenga acotada.
- Pautas de estilo: es imprescindible ceñirse al formato para la escritura de programas en C++ que se propone en esta asignatura. Algunos de los principales criterios de estilo se describirán brevemente en la sección 4 de esta práctica.
- Lectura y escritura de ficheros de texto en C++.
- Compilación de programas utilizando `make` [4, 5].

2. Contenedores estándar

Un contenedor es un objeto que almacena una colección de otros objetos (sus elementos). Se implementan como plantillas de clase, lo que permite una gran flexibilidad en los tipos admitidos como elementos. El contenedor gestiona el espacio de almacenamiento de sus elementos y proporciona funciones miembro para acceder a ellos, ya sea directamente o a través de iteradores (objetos de referencia con propiedades similares a las de los punteros).

La STL (*Standard Template Library*) [1] de C++ proporciona una amplia variedad de contenedores [2, 3] que se pueden utilizar para almacenar y manipular datos de diferentes maneras. Estos contenedores se pueden agrupar en varios tipos principales según sus características y comportamientos.

2.1. Contenedores de secuencia

Los contenedores secuenciales implementan estructuras de datos que pueden ser accedidas de manera secuencial.

- `std::vector`: un array dinámico que permite un rápido acceso a elementos mediante índices.
- `std::list`: una lista doblemente enlazada que permite la inserción y eliminación eficiente de elementos en cualquier posición.
- `std::deque`: una cola doblemente terminada (*double-ended queue*) que permite la inserción y eliminación eficiente tanto al principio como al final.

A partir de C++11 también están disponibles `std::array` y `std::forward_list`.

2.2. Contenedores asociativos

Los contenedores asociativos se utilizan para almacenar y gestionar datos de manera que se permita un acceso eficiente y rápido a través de un mecanismo de búsqueda basado en claves. Estos contenedores asocian una clave con un valor, y facilitan la recuperación de valores utilizando la clave correspondiente en lugar de la posición física en la estructura de datos. Esta característica los hace ideales para aplicaciones en las que se requiere una búsqueda eficiente de datos.

En C++, los contenedores asociativos se pueden dividir en dos categorías principales: contenedores asociativos ordenados y contenedores asociativos desordenados (también conocidos como contenedores no ordenados). La principal diferencia entre ellos radica en cómo se almacenan y organizan los elementos.

2.2.1. Contenedores asociativos ordenados

Los contenedores asociativos ordenados mantienen sus elementos en un orden específico, generalmente determinado por una función de comparación o un criterio de orden definido por el usuario. Esto significa que los elementos se almacenan en una estructura de datos ordenada (por ejemplo, un árbol binario equilibrado) que permite una búsqueda eficiente (complejidad $O(\log n)$) y la recuperación de elementos en función de su orden.

- `std::set`: almacena valores únicos ordenados en función de un criterio de orden personalizado (por defecto, el operador `<`) [9, 10, 11].
- `std::map`: almacena pares clave-valor únicos ordenados en función de la clave [12, 13].
- `std::multiset`: similar a `std::set`, pero permite valores duplicados.
- `std::multimap`: similar a `std::map`, pero permite claves duplicadas.

2.2.2. Contenedores asociativos no ordenados (a partir de C++11)

Los contenedores asociativos desordenados no mantienen un orden específico entre los elementos almacenados. Utilizan estructuras de datos basadas en tablas hash para lograr un acceso eficiente y rápido a través de una función de hash (complejidad $O(1)$ en media y $O(n)$ en el peor de los casos). Estos contenedores no garantizan un orden particular de los elementos pero son útiles cuando la prioridad es la velocidad de acceso.

- `std::unordered_set`: almacena valores únicos utilizando una tabla hash para un acceso rápido.
- `std::unordered_map`: almacena pares clave-valor únicos utilizando una tabla hash.
- `std::unordered_multiset`: similar a `std::unordered_set`, pero permite valores duplicados.
- `std::unordered_multimap`: similar a `std::unordered_map`, pero permite claves duplicadas.

2.3. Otros contenedores

La STL también proporciona otros contenedores como: las parejas, los adaptadores y las vistas. En el caso del contenedor `std::pair` debemos tener en cuenta que se trata de un contenedor simple formado por una tupla de dos elementos de datos u objetos, denominados “primero” (`first`) y “segundo” (`second`), en ese orden fijo. Un *pair* STL se puede asignar, copiar y comparar. Los elementos asignados en un mapa (ordenado o no) son de tipo `std::pair` por defecto, donde todos los elementos `first` actúan como las claves únicas, cada una asociada con sus objetos de valor `second`.

3. Ejercicio práctico

Los contenedores asociativos son una herramienta poderosa en C++ para resolver una variedad de problemas de búsqueda y almacenamiento de datos. La elección del contenedor adecuado depende de los requisitos específicos de tu programa, incluyendo si se requieren claves únicas o duplicadas y si se necesita mantener un orden específico en los datos. En esta práctica, utilizaremos en particular los contenedores asociativos ordenados para resolver los dos ejercicios que se plantearán a continuación.

3.1. Calificación única por estudiante

Se dispone de una serie de calificaciones que los estudiantes de una asignatura han ido obteniendo en el examen de teoría y en sus posteriores recuperaciones. Las calificaciones están almacenadas en un fichero de texto plano en el que en cada línea se encuentra el *alu* del estudiante junto con la calificación correspondiente (ambos datos separados por un espacio en blanco):

```
alu0122334455 5.25
alu0101010101 4.75
alu0198765432 8.5
alu0101010101 9.25
alu0155555555 7
```

De esta forma, para cada estudiante se almacenará únicamente una nota: la máxima de entre todas las obtenidas en los exámenes realizados. Por ejemplo, para el *alu0101010101* deberíamos almacenar la calificación 9,25 puesto que es mayor que la calificación 4,75. Para el ejemplo anterior, la visualización por pantalla debería ser similar a la siguiente:

```
alu0101010101 9.25
alu0122334455 5.25
alu0155555555 7
alu0198765432 8.5
```

Teniendo en cuenta los contenedores asociativos presentados en la sección anterior, desarrolle un programa en C++ que dado un fichero de entrada con la especificación de las calificaciones, almacene los valores correspondientes y los muestre por pantalla de forma ordenada en función del *alu*.

El programa recibirá por línea de comandos el nombre del fichero de entrada:

```
./p01_single_grades grades.txt
```

El comportamiento del programa al ejecutarse en línea de comandos debiera ser similar al de los comandos de Unix. Así por ejemplo, si se ejecuta el programa sin parámetros, se debería obtener información sobre el uso correcto del programa:

```
$ ./p01_single_grades
Modo de empleo: ./p01_single_grades grades.txt
Pruebe 'p01_single_grades --help' para más información.
```

La opción `--help` en línea de comandos ha de producir que se imprima en pantalla un breve texto explicativo del funcionamiento del programa. Una información que puede ser de especial ayuda para los usuarios del programa sería precisamente el formato del fichero que contendrá las calificaciones.

Al ejecutar correctamente el programa, además de mostrar alfabéticamente el listado de los estudiantes junto con su calificación en la asignatura, se deberá proporcionar una opción para insertar elementos de forma individual y visualizar a continuación el estado final de las calificaciones.

3.2. Múltiples calificaciones por estudiante

En este caso, se dispone de una serie de calificaciones que los estudiantes de una asignatura han ido obteniendo en los diferentes elementos evaluativos de la asignatura. Las calificaciones están almacenadas en un fichero de texto plano en el que en cada línea se encuentra el *alu* del estudiante junto con la calificación correspondiente (ambos datos separados por un espacio en blanco):

```
alu0155555555 7
alu0122334455 4.75
alu0122334455 5.25
alu0101010101 9.25
alu0155555555 6.25
alu0198765432 8.5
alu0101010101 7.25
alu0155555555 8.0
alu0155555555 2.75
```

De esta forma, cada estudiante puede tener asociadas una o más calificaciones. Téngase en cuenta que ni los estudiantes tienen que aparecer ordenados en el fichero de entrada ni todos los estudiantes deben tener el mismo número de calificaciones pues no todos habrán presentado el mismo número de actividades.

En este caso, desarrolle un programa en C++ que dado un fichero de entrada con la especificación de las calificaciones, agrupe todos los valores correspondientes a un estudiante y los muestre por pantalla de forma ordenada en función del *alu*. En cada línea se

deberá mostrar el *alu* del estudiante seguido de dos puntos y, a continuación, y separado por espacios, cada una de las calificaciones del estudiante (según se han ido encontrando en el fichero de entrada). Por ejemplo, teniendo en cuenta el fichero de entrada anterior, la salida del programa debería ser análoga a la siguiente:

```
alu0101010101: 9.25 7.25
alu0122334455: 4.75 5.25
alu0155555555: 7 6.25 8.0 2.75
alu0198765432: 8.5
```

El programa recibirá por línea de comandos el nombre del fichero de entrada:

```
./p01_multiple_grades grades.txt
```

El comportamiento del programa al ejecutarse en línea de comandos debiera ser similar al de los comandos de Unix, tal y como se describió para el ejercicio anterior.

Al ejecutar correctamente el programa, además de mostrar alfabéticamente el listado de los estudiantes junto con sus correspondientes calificaciones en la asignatura, se deberá proporcionar una opción para insertar elementos de forma individual y visualizar a continuación el estado final de las calificaciones de todos los estudiantes.

4. Estilo y formato del código

Esta relación no pretende ser exhaustiva. Contiene una serie de requisitos y/o recomendaciones cuyo cumplimiento se impulsará a la hora de evaluar las prácticas de la asignatura.

- Estudien la finalidad de los especificadores de visibilidad (`public`, `private`, `protected`) de atributos y métodos. Una explicación breve y simple es ésta [6] aunque es fácil hallar muchas otras. Se debe restringir los atributos y métodos públicos a aquellos que son estrictamente necesarios para que un programa cliente utilice la clase en cuestión.
- El principio de responsabilidad única (*Single responsibility principle, SRP*) [7] es uno de los conocidos como principios “SOLID” y es posiblemente uno de los más fáciles de comprender. Dejando a un lado el resto de principios SOLID, que pueden ser más complejos, merece la pena que lean algo sobre el SRP. Ese principio es una de las razones por las que en sus programas no debiera haber una única clase “*que se encarga de todo*”. En esta referencia [8] tienen un sencillo ejemplo (el de la clase `Person` que puede servir para ilustrar el SRP, pero no es difícil encontrar múltiples

ejemplos del SRP. Tengan en cuenta que es un principio de orientación a objetos, y por lo tanto independiente del lenguaje de programación: pueden encontrar ejemplos de código en C++, Java, Python u otros lenguajes.

- Un programa simple en C++ debiera incluir al menos 3 ficheros:
 - `X.h` para la definición de la clase `X`
 - `X.cc` para la implementación de la clase `X`
 - `ClienteX.cc` para el programa “cliente” que utiliza la clase `X`
- Para evitar la doble inclusión de un mismo fichero (habitualmente un fichero de cabecera, *header*) en un código fuente hay básicamente dos posibilidades: las *include guards* [17] y el uso de la directiva `#pragma once` [18]. Es conveniente que conozcan ambas alternativas, el funcionamiento de cada una y que tengan en cuenta que la directiva `#pragma once`, aunque soportada por la mayor parte de los compiladores de C++, no es estándar. Para más información sobre este asunto pueden leer esta discusión de StackOverflow [19].
- Ejemplo de comentario de cabecera (inicial) para (todos) los ficheros de un proyecto (práctica) de CyA

```
// Universidad de La Laguna
// Escuela Superior de Ingeniería y Tecnología
// Grado en Ingeniería Informática
// Asignatura: Computabilidad y Algoritmia
// Curso: 2º
// Práctica 1: Contenedores asociativos
// Autor: Nombre y Apellidos
// Correo: aluXXXXX@ull.edu.es
// Fecha: 12/09/2023
// Archivo cya-P01-single-grades.cc: programa cliente.
//      Contiene la función main del proyecto que usa las clases X e Y
//      para ... (indicar brevemente el objetivo)
// Referencias:
//      Enlaces de interés
//
// Historial de revisiones
//      12/09/2023 - Creación (primera versión) del código
```

- Además de la información anterior sobre el autor, la fecha y la asignatura, la cabecera también debería contener al menos una breve descripción sobre lo que hace el código incluyendo los objetivos del proyecto en general, las estructuras de datos utilizadas, así como un listado de modificaciones (bug fixes) que se han ido introduciendo en el código. El fichero de cabecera sería básicamente el mismo para todos los ficheros (`*.cc`, `*.h`) del proyecto, con la salvedad de que varía la descripción del contenido del fichero y su finalidad.

- El utilizar una estructura de directorios con nombres estándar (`bin`, `lib`, `src`, `config`, etc.) para los proyectos de desarrollo de software es una buena práctica cuando se trata de proyectos que involucran un elevado número de ficheros. No obstante, para la mayoría de prácticas de esta asignatura, que no contienen más de una decena de ficheros, colocar todos los ficheros de cada práctica en un mismo directorio (con nombre significativo *CyA/practicas/p01_single_grades*, por ejemplo) es posiblemente más eficiente a la hora de localizar y gestionar los ficheros de cada una de las prácticas (proyectos).
- En el directorio de cada práctica, como se ha indicado anteriormente, debería haber un fichero **Makefile** de modo que el comportamiento del programa `make` fuera tal que al ejecutar:

```
1 make clean
```

el programa dejara en el directorio en que se ejecuta solamente los ficheros conteniendo código fuente (ficheros `*.h` y `*.cc`) y el propio **Makefile**. En estos breves tutoriales [4, 5] se explica de forma incremental cómo construir un fichero **Makefile** para trabajar con la compilación de proyectos simples en C++.

- A continuación enumeraremos algunas cuestiones relativas al formato del código:
 1. A ambos lados de un operador binario han de escribir un espacio:
$$a + b$$

En lugar de:
$$a+b$$
 2. SIEMPRE después de una coma, ha de ir un espacio.
 3. Se debe indentar el código usando espacios y NO tabuladores. Cada nivel de indentación ha de hacerse con 2 espacios.
 4. TODO identificador (de clase, de método, de variable, ...) ha de ser significativo. No se pueden usar identificadores de un solo caracter salvo para casos concretos (variable auxiliar para un bucle o similar).
 5. TODO fichero de código de un proyecto ha de contener un prólogo con comentarios de cabecera donde se indique al menos: Autor, datos de contacto, Fecha, Asignatura, Práctica, Finalidad del código. Véase el ejemplo de cabecera proporcionado en este documento.
 6. Asimismo todos los métodos y clases han de tener al menos un mínimo comentario que documente la finalidad del código.
 7. No comentar lo obvio. No se trata de comentar por comentar, sino de aclarar al lector la finalidad del código que se escribe [15].

8. Como regla de carácter general, el código fuente de los programas no debiera contener de forma explícita constantes. En lugar de escribir

```
double balance[10];
```

Es preferible:

```
const int SIZE_BALANCE = 10;
...
double balance[SIZE_BALANCE];
```

Para el caso de las constantes numéricas, las únicas excepciones a esta regla son los valores 0 y 1. Tengamos en cuenta que las constantes no son solamente numéricas: puede haber constantes literales, de carácter o de otros tipos. Así en lugar de escribir:

```
myString.find(' ');
```

Es preferible:

```
const char SPACE = ' ';
...
myString.find(SPACE);
```

9. Cuando se programa un código encadenando múltiples sentencias if-else, ello suele ser síntoma de una mala práctica y, en efecto, debería buscarse alguna alternativa. En este sentido les recomendamos que revisen las diferentes sugerencias que se realizan al hilo de esta discusión [\[16\]](#).
10. Como regla general, se espera que todo el código fuente siga la guía de estilo de Google para C++ [\[14\]](#). Dentro de esa guía, y para comenzar, prestaremos particular atención a los siguientes aspectos:
- Formateo del código (apartado *Formatting* en [\[14\]](#))
 - Comentarios de código de diverso tipo (apartado *Comments* en [\[14\]](#))
 - Nominación de identificadores, clases, ficheros, etc. (apartado *Naming* en [\[14\]](#))

5. Criterios de evaluación

Se señalan a continuación los aspectos más relevantes (la lista no es exhaustiva) que se tendrán en cuenta a la hora de evaluar esta práctica:

- Se valorará que el alumnado haya realizado, con anterioridad a la sesión de prácticas, y de forma efectiva, todas las tareas propuestas en este guión.
- También se valorará que, con anterioridad a la sesión de prácticas, el alumnado haya revisado los documentos que se enlazan desde este guión.
- Paradigma de programación orientada a objetos: se valorará que el alumnado identifique adecuadamente los contenedores estándar que puede utilizar en cada uno de los casos.
- Paradigma de modularidad: se valorará que el programa se haya escrito de modo que las diferentes funcionalidades que se precisen hayan sido encapsuladas en métodos concretos cuya extensión textual se mantuviera acotada.
- Se valorará que el código desarrollado siga el formato propuesto en esta asignatura para la escritura de programas en C++.
- Capacidad del programador(a) de introducir cambios en el programa desarrollado.

Si el alumnado tiene dudas respecto a cualquiera de estos aspectos, debiera acudir al foro de discusiones de la asignatura para plantearlas allí. Se espera que, a través de ese foro, el alumnado intercambie experiencias y conocimientos, ayudándose mutuamente a resolver dichas dudas. También el profesorado de la asignatura intervendrá en las discusiones que pudieran suscitarse, si fuera necesario.

Referencias

- [1] Standard Template Library en la Wikipedia: https://en.wikipedia.org/wiki/Standard_Template_Library#Containers
- [2] STL Containers: <https://cplusplus.com/reference/stl/>
- [3] STL Containers: <https://en.cppreference.com/w/cpp/container>
- [4] Makefile Tutorial: <http://www.cs.colby.edu/maxwell/courses/tutorials/maketutor>
- [5] C++ Makefile Tutorial: <https://makefiletutorial.com>
- [6] Difference between private, public and protected inheritance: <https://stackoverflow.com/questions/860339/difference-between-private-public-and-protected-inheritance>
- [7] Single-Responsability Principle (SRP): https://en.wikipedia.org/wiki/Single_responsibility_principle
- [8] Example of the SRP: <https://stackoverflow.com/questions/10620022/what-is-an-example-of-the-single-responsibility-principle>
- [9] STL sets: <http://www.cplusplus.com/reference/set/set/>
- [10] STL set example with default sorting criteria: <https://thispointer.com/stdset-tutorial-part-1-set-usage-details-with-default-sorting-criteria/>
- [11] STL set example with user defined classes: <https://thispointer.com/using-stdset-with-user-defined-classes/>
- [12] STL maps: <https://cplusplus.com/reference/map/map/>
- [13] STL map example: <https://www.educba.com/c-plus-plus-map/>
- [14] Google C++ Style Guide, <https://google.github.io/styleguide/cppguide.html>
- [15] Commenting code: <https://www.cs.utah.edu/~germain/PPS/Topics/commenting.html>
- [16] If-else-if chains or multiple-if: <https://stackoverflow.com/questions/25474906/which-is-better-if-else-if-chain-or-multiple-if>
- [17] Include guard: https://en.wikipedia.org/wiki/Include_guard
- [18] Pragma once: https://en.wikipedia.org/wiki/Pragma_once
- [19] Pragma once vs. Include guards: <https://stackoverflow.com/questions/1143936/pragma-once-vs-include-guards>